

ROS2 실습 요약본

1. ROS2 CLI

1.1 ROS2 CLI (Command Line Interface)

```
ros2 [verbs] [sub-verbs] [options] [arguments]
```

- **verbs** : 동작을 지정하며, 수행할 작업의 유형을 나타냄. topic, service, node 등이 올 수 있음
- **sub-verbs** : 특정 동작에 대한 세부 동작(sub-verb)을 지정함. Verbs가 topic인 경우 pub, echo, list, hz 등이 올 수 있음
- **options** : 명령어의 실행 방식을 설정하는 추가 파라미터. -h, -r, --node-name, --qos, --ros-args 등이 올 수 있음
- **arguments** : 실행할 때 필요한 인수를 지정함. 특정 노드의 이름이나 토픽의 이름, 서비스 이름 등이 올 수 있음
- -h 옵션(**ros2 -h**)을 이용하면 verbs, sub-verbs, option등에 대하여 더 자세히 알 수 있음
 - **ROS2 CLI 실행 명령어 + arguments**

ros2cli + [verbs]	[arguments]	기능
ros2 run	<package> <executable>	특정 패키지의 노드 실행
ros2 launch	<package> <launch file>	런치 파일을 통해 복수 노드 실행

- **주요 명령어 (Verbs/ Sub-verbs)**

ros2cli + [verbs]	[sub-verbs]	기능
ros2 pkg	create	새로운 ROS2 패키지 생성
	executables	executables 지정 패키지의 실행 파일 목록 출력
	list	list 사용 가능한 패키지 목록 출력
	prefix	prefix 지정 패키지의 저장 위치 출력
	xml	xml 지정 패키지의 패키지 정보 파일(xml) 출력
ros2 node	info	실행 중인 노드 중 지정한 노드의 정보 출력
	list	실행 중인 모든 노드의 목록 출력

ros2 topic	bw	지정 토픽의 대역폭 측정
	delay	지정 토픽의 지연시간 측정
	echo	지정 토픽의 데이터 출력
	find	지정 타입을 사용하는 토픽 이름 출력
	hz	지정 토픽의 주기 측정
	info	지정 토픽의 정보 출력
	list	사용 가능한 토픽 목록 출력
	pub	지정 토픽의 토픽 발행
	type	지정 토픽의 토픽 타입 출력
ros2 service	call	지정 서비스의 서비스 요청 전달
	find	지정 서비스 타입의 서비스 출력
	list	사용 가능한 서비스 목록 출력
	type	지정 서비스의 타입 출력
ros2 action	info	지정 액션의 정보 출력
	list	사용 가능한 액션 목록 출력
	send_goal	지정 액션의 액션 목표 전송
ros2 interface	list	사용 가능한 모든 인터페이스 목록 출력
	package	특정 패키지에서 사용 가능한 인터페이스 목록 출력
	packages	인터페이스 패키지들의 목록 출력
	proto	지정 패키지의 프로토타입 출력
	show	지정 인터페이스의 데이터 형태 출력
ros2 param	delete	지정 파라미터 삭제
	describe	지정 파라미터 정보 출력
	dump	지정 파라미터 저장
	get	지정 파라미터 읽기
	list	사용 가능한 파라미터 목록 출력
	set	지정 파라미터 쓰기
ros2 bag	info	저장된 rosbag 정보 출력
	play	rosbag 기록
	record	rosbag 재생

◦ 보조 명령어(확장 기능 관리)

ros2cli + [verbs]	[sub-verbs] (options)	기능
ros2 extensions	(-a) (-v)	ros2cli의 extension 목록 출력 (ros2cli개발용으로 사용, 일반적 사용 x)
ros2 extension_points	(-a) (-v)	ros2cli의 extension point 목록 출력 (ros2cli개발용으로 사용, 일반적 사용 x)
ros2 daemon	start	daemon 시작
	status	daemon 상태 보기
	stop	daemon 정지
ros2 multicast	receive	multicast 수신
	send	multicast 전송
ros2 doctor	hello (-r) (-rf) (-iw)	ROS 설정 및 네트워크, 패키지 버전, rmw 미들웨어 등과 같은 잠재적 문제를 확인하는 도구
ros2 wtf	hello (-r) (-rf) (-iw)	doctor와 동일함 (ros2 doctor의 alias) (WTF: Where's The Fire)
ros2 lifecycle	get	라이프사이클 정보 출력
	list	지정 노드의 사용 가능한 상태 전이 목록 출력
	nodes	라이프사이클을 사용하는 노드 목록 출력
	set	라이프사이클 상태 전환 트리거
ros2 component	list	실행 중인 컨테이너와 컴포넌트 목록 출력
	load	지정 컨테이너 노드의 특정 컴포넌트 실행
	standalone	표준 컨테이너 노드로 특정 컴포넌트 실행
	types	사용 가능한 컴포넌트들의 목록 출력
	unload	지정 컴포넌트의 실행 중지
ros2 security	create_key	보안키 생성
	create_keystore	보안키 저장소 생성
	create_permission	보안 허가 파일 생성
	generate_artifacts	보안 정책 파일을 이용하여 보안키 및 보안 허가 파일 생성
	generate_policy	보안 정책 파일(policy.xml) 생성
	list_keys	보안키 목록 출력

- 자주 사용하는 CLI 명령어 예시

```
# 패키지 내 노드 실행
ros2 run <패키지명> <노드명>

# 런치 파일 실행
ros2 launch <패키지명> <런치파일명>

# 토픽 목록 조회
ros2 topic list

# 토픽 데이터 수신 (모니터링)
ros2 topic echo <토픽명>

# 노드 목록 조회
ros2 node list

# 서비스 호출
ros2 service call <서비스명> <서비스타입> "{파라미터: 값}"
```

- **alias**

- ~/.bashrc 파일에 자주 사용하는 ROS2 CLI 명령어를 단축 명령어로 지정 가능
- bashrc 파일 수정적용 후 source ~/.bashrc를 통해 현재 세션에 설정 적용
- 지정해둔 단축키 이용하여 명령어 빠르게 사용 가능

- **ROS arguments**

- `ros2 run`, `ros2 launch` 같은 실행 명령어에 **추가 설정을 부여**하는 옵션
- 실행 시 `-ros-args` 뒤에 붙여 사용
- **자주 사용하는 arguments**

인자	설명
<code>-r __node:=<노드이름></code>	노드 이름 변경
<code>-r __ns:=<네임스페이스></code>	네임스페이스 설정
<code>-r <기존토픽명>:=<새토픽명></code>	토픽 리맵

<code>-p <파라미터이름>:=<값></code>	파라미터 직접 지정
<code>--params-file <경로.yaml></code>	YAML 파일로 파라미터 일괄 설정

◦ 예제 1)

```
# 노드 이름을 my_turtle로, 네임스페이스는 /tutorial로 설정
ros2 run turtlesim turtlesim_node \
  --ros-args \
  -r __node:=my_turtle \
  -r __ns:=/tutorial
```

◦ 예제 2)

```
# yaml 파일로 파라미터 재설정
turtlesim_node:
ros__parameters:
  background_r: 150
  background_g: 200
  background_b: 250
```

```
ros2 run turtlesim turtlesim_node \
  --ros-args --params-file params.yaml
```

1.2 ROS2 신규 CLI 작성법

1. 목적

- `ros2 env` 처럼 기존에 없는 **새로운 CLI 명령어(verb)**를 만들고 실행
- **ros2cli 확장 구조**를 활용하여 **나만의 명령어** 추가 가능

2. 기본 폴더 구조

```
ros2env/
├── command/
│   └── env.py    # 메인 명령어 엔트리 포인트
├── verb/
│   ├── list.py  # 서브 명령어 list
│   ├── set.py   # 서브 명령어 set
│   └── init.py  # Verb 등록
```

```

├── api/
│   └── init.py    # 환경변수 처리 로직
└── setup.py      # entry_points 등록

```

3. 주요 구성요소 설명

1) **command/env.py**

- 메인 명령어 진입점: `ros2 env`
- `EnvCommand` 클래스가 핵심
- 주요 메서드:
 - `add_arguments()` : 서브명령어 연결
 - `main()` : 실행 진입점

```

class EnvCommand(CommandExtension):
    def add_arguments(self, parser, cli_name):
        ...
    def main(self, *, parser, args):
        ...

```

2) **verb/list.py**

- `ros2 env list` 명령어 구현
- 환경 변수 종류별 출력 기능

```

class ListVerb(VerbExtension):
    def add_arguments(self, parser, cli_name):
        parser.add_argument("-a", "--all", action="store_true", help="...")

    def main(self, *, args):
        if args.all:
            print(get_all_env_list())

```

3) **verb/set.py**

- `ros2 env set <VAR> <VAL>` 형식으로 환경변수 설정
- `set_ros_env(name, value)` 사용

```
class SetVerb(VerbExtension):
    def add_arguments(self, parser, cli_name):
        parser.add_argument("env_name")
        parser.add_argument("value")

    def main(self, *, args):
        set_ros_env(args.env_name, args.value)
```

4) `api/__init__.py`

- 실제 환경 변수 처리 함수 구현

```
def get_ros_env_list():
    return f"ROS_DISTRO={os.getenv('ROS_DISTRO')}"

def set_ros_env(name, value):
    os.environ[name] = value
    return f"{name} set to {value}"
```

5) `setup.py`

- CLI 확장을 등록하는 핵심 파일

```
entry_points={
    'ros2.cli.command': [
        'env = ros2env.command.env:EnvCommand',
    ],
    'ros2env.verb': [
        'list = ros2env.verb.list:ListVerb',
        'set = ros2env.verb.set:SetVerb',
    ],
}
```

4. 실행 예시

```
# 환경 변수 목록 출력
ros2 env list --all
```

```
# ROS_DISTRO 환경변수 설정
ros2 env set ROS_DISTRO humble
```

2. ROS2 심화 기능1

2.1 Intra-process communication

: 동일 프로세스 내 노드 간 통신 최적화 방식

1. 개념 요약

- ROS2에서는 기본적으로 **노드 간 메시지 통신 시 메모리 복사**가 발생
- 여러 노드가 **동일한 프로세스 내에서 실행**될 경우에도
→ 기본 설정대로라면 DDS를 통해 **메모리 복사 후 전송**
- **Intra-Process Communication(IPC)**은 이 중복된 복사를 **줄여주는 기술**

2. 일반 통신 구조의 문제점

- 퍼블리셔 → DDS 버퍼로 복사 → 구독자 수신 → 다시 복사
- 특히 **이미지, LiDAR** 등 대용량 데이터 전송 시 성능 저하
- 예시)
 - 퍼블리셔가 이미지 생성 → DDS로 복사
 - 구독자 3명이면 3번 추가 복사(총 4번의 복사 발생)

3. 해결책 : Intra-Process방식

- 핵심 원리: **Zero-Copy**
- 퍼블리셔가 생성한 메시지를 **공유 메모리에 저장**
- 구독자는 **복사 없이 해당 메모리 주소 참조**
- 조건 : 퍼블리셔와 구독자가 **같은 프로세스 내(동일 PID)**에 있어야 함

4. 구조 비교

구분	일반 DDS 통신	Intra-Process
프로세스	서로 다름	동일
메모리 복사	다수 발생	없음 (참조 방식)
속도	상대적으로 느림	매우 빠름
사용 예	분산 로봇 시스템	단일 로봇 내 처리 최적화

5. 프로그래밍 시 적용방법

- `intra_process_comms=True` 옵션을 사용
- 예: `rclcpp::NodeOptions().use_intra_process_comms(true)`

2.2 DDS의 QoS

1. QoS(Quality of Service)란?

- QoS는 ROS2의 통신 품질 옵션을 의미
- DDS(Data Distribution Service) 기반의 통신에서
퍼블리셔 ↔ 서브스크라이버 사이의 데이터 전달 방식을 설정하는 기술
- 속도, 신뢰성, 수명, 보관 주기 등 다양한 조건 설정 가능

2. 주요 QoS 설정 항목

항목명	설명	주요 값
History	데이터를 몇 개까지 저장할지	<code>KEEP_LAST</code> , <code>KEEP_ALL</code> + <code>depth</code>
Reliability	데이터 전송의 신뢰성	<code>BEST_EFFORT</code> , <code>RELIABLE</code>
Durability	구독자 생성 전 메시지 보존 여부	<code>VOLATILE</code> , <code>TRANSIENT_LOCAL</code>
Deadline	지정 주기 내 미수신 시 경고 발생	<code>deadline_duration</code> 설정
Lifespan	메시지의 유효 시간	<code>lifespan_duration</code> 설정
Liveliness	노드 생존 확인 방식	<code>AUTOMATIC</code> , <code>MANUAL_BY_NODE</code> , <code>MANUAL_BY_TOPIC</code>

3. 각 항목 요약설명

1) History

- 메시지를 몇 개나 보관할지 설정

- `KEEP_LAST` : 마지막 n개만 유지
- `KEEP_ALL` : 가능한 모든 메시지 유지

2) Reliability

- 메시지 **전송 신뢰성**
- `BEST_EFFORT` : 빠르지만 일부 손실 허용
- `RELIABLE` : 손실 없이 보장 (재전송)

3) Durability

- 구독자가 나중에 생겨도 메시지를 받을지
- `VOLATILE` : 최신 메시지만 전달
- `TRANSIENT_LOCAL` : 이전 메시지도 보관 후 전달

4) Deadline

- 설정된 주기 내 메시지를 못 받으면 콜백 실행
- 주기 설정: `rclpy.duration.Duration(seconds=X)`

5) Lifespan

- 메시지가 유효한 시간 설정 (이후는 자동 폐기)

6) Liveliness

- 노드가 살아있는지 감지
- 설정값: `AUTOMATIC` , `MANUAL_BY_NODE` , `MANUAL_BY_TOPIC`

4. QoS Profile 객체 설정 예시

```
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

custom_qos = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=10,
    reliability=QoSReliabilityPolicy.RELIABLE
)
```

5. 기본 QoS 프로파일(ROS Middleware QoS Profiles)

용도	Reliability	History	Depth	Durability
Default	RELIABLE	KEEP_LAST	10	VOLATILE
Sensor Data	BEST_EFFORT	KEEP_LAST	5	VOLATILE
Parameters	RELIABLE	KEEP_LAST	1000	VOLATILE
Action Status	RELIABLE	KEEP_LAST	1	TRANSIENT_LOCAL

• DDS & QoS

항목	DDS (Data Distribution Service)	QoS (Quality of Service)
개념	- ROS2 기본 통신 미들웨어로 중앙 서버 없이 분산 네트워크에서 데이터를 교환 - DDS는 ROS2의 기본 통신 프로토콜	- DDS 통신의 품질을 조정하는 설정으로, 메시지 신뢰성, 내구성, 저장 개수 등을 관리 - DDS를 통해 통신 품질을 조정하는 방식-QoS 설정에 따라 DDS의 동작 방식이 달라지며 각 애플리케이션에 맞는 성능을 최적화 - 네트워크 환경과 응용 프로그램의 요구사항에 따라 적절한 QoS를 설정해야 함
특징	- 실시간 데이터 교환을 위한 미들웨어 표준, ROS2의 통신기반이 되는 핵심 기술	- QoS 주요 설정 6종류:Reliability(신뢰성), Durability(내구성), History(데이터 저장 개수), Lifespan(수명), Deadline(데드라인), Liveliness(활성 상태)
기타	- Publisher-subscriber 모델 - Brokerless (중앙 서버 없음) - 자동 발견(Discovery) : 네트워크상의 노드들이 서로를 자동으로 인식하고 연결 - 신뢰성 & 확장성 : 다양한 QoS 설정을 통해 신뢰성과 성능을 조절	- BEST_EFFORT + VOLATILE : 카메라 영상 스트리밍 (실시간성 우선, 일부 프레임 손실 가능) - RELIABLE + TRANSIENT_LOCAL : 로봇 센서 데이터 (정확한 수신 보장, 최신 데이터 유지) - KEEP_LAST(10) + LIFESPAN(5s) : 5초 동안 최신 10개의 데이터를 유지하는 센서 데이터

2.3 Topic, Service, Action의 QoS 설정

항목	기본 QoS 설정	설명	핵심 속성
Topic	RELIABLE, KEEP_LAST, depth=10, VOLATILE	대부분의 퍼블리셔/서브스크라이버는 기본 설정으로 동작	속도 or 신뢰성 중심
Service	qos_profile_services_default (RELIABLE + VOLATILE)	요청/응답 구조, 신뢰성 보장 필수	신뢰성 필수 (RELIABLE)

Action	여러 QoS 설정을 조합 - goal, result, cancel 요청: <code>qos_profile_services_default</code> - feedback 퍼블리셔: <code>QoSProfile(depth=10)</code> - status 퍼블리셔: <code>qos_profile_action_status_default</code>	Action은 내부적으로 Topic + Service 혼합 구조이기 때문에 각 역할마다 QoS가 다름	복합적 QoS 조합 사용됨
---------------	---	--	----------------

3. ROS2 심화 기능 2

3.1 ROS2 component

: 여러 노드를 하나의 프로세스 내에서 실행시켜 자원낭비를 줄이고 통신효율 향상을 도모하는 구조

- 일반적으로 노드는 각자 별도의 프로세스로 실행되며, 노드 간 통신 시 DDS를 거쳐야 함 → 성능 저하
- Component 방식은 하나의 컨테이너 프로세스 안에 여러 노드를 포함시켜 **Intra-Process Communication** 가능

1. 구성 요소

용어	설명
Component Node	<code>.so</code> 형태로 빌드된 재사용 가능한 노드
Container	여러 component 노드를 실행할 수 있는 공용 런타임 프로세스
Composition	여러 노드를 조합하여 하나의 프로세스에 올리는 방식

2. CLI 실습 명령어

목적	명령어	설명
컴포넌트 목록 확인	<code>ros2 component types</code>	사용 가능한 component 노드 타입 출력
컨테이너 실행	<code>ros2 run rclcpp_components component_container</code>	빈 컨테이너 실행
컨테이너 목록 확인	<code>ros2 component list</code>	실행 중인 컨테이너와 포함된 컴포넌트 노드 목록 출력
컴포넌트 추가 로드	<code>ros2 component load <container> <package> <plugin></code>	실행 중인 컨테이너에 노드 추가
컴포넌트 언로드	<code>ros2 component unload <container> <node_id></code>	해당 노드 제거

단독 실행	<code>ros2 component standalone</code> <code><package> <plugin></code>	단일 컨테이너에서 특정 컴포넌트 실행
-------	---	----------------------

3. 실습 예시 플로우

```
# 1. 컨테이너 실행 (rclcpp 기준)
ros2 run rclcpp_components component_container

# 2. 사용 가능한 컴포넌트 확인
ros2 component types

# 3. 컴포넌트 로드
ros2 component load /ComponentManager demo_nodes_cpp talker

# 4. 컴포넌트 언로드
ros2 component unload /ComponentManager <node_id>

# 5. 단독 컨테이너 실행 예시
ros2 component standalone demo_nodes_cpp talker
```

4. 장점

- 메모리 복사 최소화 (Intra-process communication)
- 런타임에서 노드 추가/제거 가능
- 개발한 노드를 공유 라이브러리처럼 재사용

3.2 RQt Plugin

: ROS2 데이터를 시각적으로 표현하거나, 노드 상태를 디버깅할 수 있도록 도와주는 GUI기반 툴

- Qt 기반의 플러그인 시스템으로 구성됨
- ROS1부터 존재하며 ROS2에서도 계속 사용됨
- `rqt_graph`, `rqt_plot`, `rqt_console`, `rqt_reconfigure` 등이 대표적

1. 주요 RQt Plugin 종류

플러그인	설명
<code>rqt_graph</code>	노드, 토픽 간 연결 관계 시각화

rqt_plot	실시간 데이터(토픽 값) 그래프 표시
rqt_console	로그 메시지 확인
rqt_logger_level	노드별 로그 레벨 실시간 변경
rqt_reconfigure	노드 파라미터 실시간 변경 (dynamic reconfigure용)
rqt_bag	rosvbag 데이터 시각화 및 재생
rqt_image_view	이미지 토픽을 화면에 표시

2. 실행 방법

```
# RQt 전체 실행
rqt
```

```
# 특정 plugin만 실행
# 예: rqt --perspective-plugin rqt_graph
rqt --perspective-plugin <plugin_name>
```

3. 실습 예시

```
# 1. turtlesim 실행
ros2 run turtlesim turtlesim_node

# 2. 키보드 제어 실행
ros2 run turtlesim turtle_teleop_key

# 3. rqt_graph 실행
rqt_graph
```

• 결과

- 노드와 토픽 간 연결 시각화 가능
- `/turtle1/cmd_vel` 토픽을 통해 `turtle_teleop_key → turtlesim_node` 흐름 확인

• 장점

- 노드/토픽 구조를 **시각화**해 디버깅과 구조 분석에 매우 유용
- 데이터 흐름을 **실시간 추적**

- 초보자도 GUI 기반으로 빠르게 파악 가능

4. ROS2 심화 기능2

4.1 Lifecycle Node

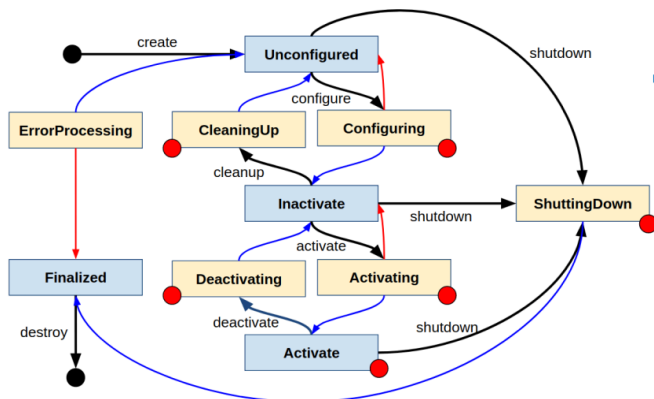
: ROS2에서 노드의 상태(state)를 명확히 관리하기 위한 노드 설계 방식

→ 시스템 상태 전이(State Transition)를 통해 제어흐름을 명확히 표현 가능

1. 주요 상태(State)

상태 이름	설명
unconfigured	노드가 아직 설정되지 않음
inactivate	설정은 되었지만 실행 전 상태
activate	노드가 활성화되어 동작 중
finalized	노드 종료 상태
errorprocessing	에러 상태

2. 상태 전이(Transition)



■ 노드의 상태와 상태 전환(Transition)

- 파란 박스: 주요 상태
- 노란 박스: 전환 상태
- 검정색 화살표: 전환을 나타냄
- 파란색 화살표: 전환 성공 시 주요 상태의 변화
- 빨간색 화살표: 전환 실패 시 주요 상태의 변화
- 빨간색 작은 원: 에러가 발생할 수 있는 상태

이벤트	전이 설명
configure	unconfigured → inactivate
activate	inactivate → activate
deactivate	activate → inactivate
cleanup	inactivate → unconfigured
shutdown	어떤 상태에서든 finalized 로 종료

3. 실습 예시

```
# 1. lifecycle_talker 실행
ros2 run demo_nodes_cpp lifecycle_talker

# 2. 상태 확인
ros2 lifecycle get /lifecycle_talker

# 3. 상태 전이 명령
ros2 lifecycle set /lifecycle_talker configure
ros2 lifecycle set /lifecycle_talker activate
ros2 lifecycle set /lifecycle_talker deactivate
ros2 lifecycle set /lifecycle_talker cleanup
ros2 lifecycle set /lifecycle_talker shutdown
```

4. 장점

- **명확한 상태 기반 제어** → 안정적인 시스템 관리
- **고장 대비 처리 및 자동 복구 로직** 구성에 유리
- 실시간 제어, 로봇 하드웨어 연동 시 상태 기반 로직 필수

4.2 ROS2 security

: DDS-Security 표준 기반으로, Ros2 노드 간 통신을 **암호화, 인증, 접근제어**하는 기능 제공

1. 환경 변수

- ROS_SECURITY_KEYSTORE** : 보안설정 파일을 보관하는 폴더를 지정.
demo_keystore
- ROS_SECURITY_ENABLE** : 보안 설정의 On/Off 기능으로 true/false 형태로 설정.
Default는 false
- ROS_SECURITY_STRATEGY** : 보안 설정 방법. Enforce로 설정하면 보안 설정 파일이 없는 메시지 통신은 금지, Permissive의 경우 비보안 참여자로 참석시킴

2. 보안 기능 구성요소

기능	설명
----	----

Authentication	노드의 신원 확인 (노드 인증)
Access Control	누가 어떤 토픽/서비스/액션에 접근 가능한지 제어
Encryption	메시지 및 데이터의 암호화
Logging	보안 이벤트 기록 및 감사 기능

2. 실습 준비 및 설정

a. SROS2 패키지 설치

```
sudo apt install ros-<distro>-sros2
```

b. 키 생성

```
# 워크스페이스 내에서 인증키 생성
ros2 security create_keystore my_keys
ros2 security create_key my_keys /my_node
```

c. 정책 생성

- 접근 가능한 리소스 명시

3. 실행 시 보안 적용

```
export ROS_SECURITY_KEYSTORE=~/.ros2_ws/my_keys
export ROS_SECURITY_ENABLE=true
export ROS_SECURITY_STRATEGY=Enforce

ros2 run demo_nodes_cpp talker
```

4. 장점

- DDS 미들웨어 보안 기능 활용 (RFC 기반)
- 로봇 시스템의 **산업용/군사/의료 환경**에서도 적용 가능
- 무단 접근 차단 및 데이터 유출 방지

• Lifecycle/ Security 핵심요약

항목	Lifecycle	Security
목적	노드 상태 전이 제어	노드 간 통신 보안 관리

주요 기능	상태 기반 activate/deactivate 관리	인증, 암호화, 접근제어
실습 명령	<code>ros2 lifecycle</code>	<code>ros2 security create_*</code>
적용 예	하드웨어 연결 상태 전이	외부 통신 제한, 암호화 전송

4.3 tf2란?

- ROS2에서 다양한 좌표 프레임 간의 관계 및 변환을 추적하는 핵심 라이브러리
- 로봇 시스템에는 일반적으로 다수의 **3D 좌표 프레임**이 존재
(예: `world`, `base_link`, `head`, `gripper`, `camera`)
- tf2를 통해 다음과 같은 질문에 답할 수 있음
 - "5초 전 world frame에서 head는 어디에 있었는가?"
 - "Gripper의 물체는 base에 비해 어떤 위치인가?"
 - "Map frame에서 base frame의 포즈는?"

2. 주요 활용 분야

활용 분야	설명
로봇 위치 추적	<code>map</code> → <code>odom</code> → <code>base_link</code> 변환을 통해 위치 확인
센서 정렬	LiDAR, 카메라, IMU 데이터를 로봇 본체 기준으로 변환
로봇 팔 제어	관절 간 변환으로 손끝 위치 계산
드론 위치 보정	GPS → map 또는 base 변환 처리

3. World Frame과 그 종류

프레임	설명
<code>map</code>	SLAM, Localization의 기준 좌표계
<code>odom</code>	바퀴 기반 이동 시 사용, drift 존재
<code>world</code>	시뮬레이터(Gazebo) 상의 절대 좌표계
<code>earth</code>	GPS 기반 전역 좌표계
<code>base_link</code>	로봇 본체 기준 프레임

4. TF2 개념정리

개념	설명
Frame	좌표계 (예: <code>map</code> , <code>odom</code> , <code>base_link</code> , <code>camera_link</code>)
Transform	프레임 간의 회전 + 이동 정보

TF Tree	여러 프레임이 계층적으로 연결된 트리 구조
Broadcaster	변환 정보를 주기적으로 발행하는 노드
Listener	발행된 프레임 정보를 구독해 변환 계산

5. tf2 시각화 도구

도구	설명
<code>view_frames</code>	현재 tf에 존재하는 모든 프레임과 그 연결 관계를 시각화해주는 도구 (좌표계 지도)
<code>tf2_echo</code>	ROS를 통해 브로드캐스트된 두 프레임 간 변환 정보 출력
<code>rviz2</code>	프레임을 3D로 시각화 (X:빨간색, Y:초록색, Z:파란색)

4.4 URDF(Universal Robot Description Format)

: 로봇의 물리적 구성과 구조를 기술하기 위한 XML 기반 포맷(.urdf)

- 로봇의 링크(몸체)와 조인트(관절)들을 정의하여 로봇 모델을 시뮬레이션/시각화/제어할 수 있게 함
- RViz, Gazebo, MoveIt 등과 함께 사용됨

1. 기본 구성 요소

- 링크(link)** : 로봇의 물리적인 부분을 나타내며, 고체 물체로 이해할 수 있음. 각 링크는 모양, 관성, 마찰 특성 등과 같은 속성들을 가짐.
- 조인트(joint)** : 두 링크를 연결하여 로봇이 움직일 수 있게 하는 연결점. 회전 운동이나 직선 운동을 정의할 수 있음
- 재질 및 텍스처** : 로봇의 각 링크에 적용할 재질이나 텍스처를 정의하여 외형을 시뮬레이션에서 표현할 수 있음
- 센서 및 액추에이터** : URDF 파일을 확장하여 센서나 액추에이터와 같은 로봇의 기능적 요소들을 정의할 수 있음

요소	설명
<code><robot></code>	URDF 파일의 루트 태그
<code><link></code>	로봇의 하나의 물리적 부위 (ex. base, arm, sensor 등)
<code><joint></code>	두 링크를 연결하는 관절 (회전, 직선 등)
<code><origin></code>	부모 링크 기준으로 자식 링크가 어느 위치에 붙는지 설정 (xyz, rpy)

<code><inertial></code> , <code><visual></code> , <code><collision></code>	물리 계산, 시각화, 충돌 처리용 속성들
--	------------------------

2. Joint Type종류

타입	설명
<code>fixed</code>	고정 (움직이지 않음)
<code>revolute</code>	회전 관절 (최소/최대 각도 범위 있음)
<code>continuous</code>	무한 회전 (360도)
<code>prismatic</code>	직선 이동
<code>floating</code> , <code>planar</code>	특수 목적 (3D/2D 자유도)

- 예시)

```
<joint name="gripper_extension" type="prismatic">
```

3. URDF + RViz 시각화

- 기본 흐름

1. URDF 파일 작성

- 로봇의 링크 및 조인트를 정의한 `.urdf` 또는 `.xacro` 파일 준비

2. robot_state_publisher 노드 실행

- URDF의 joint 구조를 기반으로 **TF (Transform Frame)** 자동 publish

3. joint_state_publisher 사용

- RViz에서 조인트를 조작할 수 있게 함

4. RViz 실행 및 설정

- Fixed Frame을 `base_link` 로 설정
- `RobotModel` 패널에서 URDF 구조 시각화 가능

- RViz에서 확인할 것

항목	설명
Fixed Frame	<code>base_link</code> 또는 URDF 내 최상위 프레임으로 지정
TF	프레임 트리과 좌표축이 정상적으로 보이는지 확인
RobotModel	로봇의 각 링크가 3D로 올바르게 시각화되는지 확인

• Joint State Publisher & Robot State Publisher

구분	Joint State Publisher	Robot State Publisher
역할	로봇의 모든 관절(Joint) 상태(Position 등)를 Publishing	<code>/joint_states</code> 를 받아서 링크 간 변환(TF)을 계산해 Publishing
발행하는 토픽	<code>/joint_statesSensor_msgs/msg/JointState</code>	<code>/tf</code> → 움직이는 링크 변환 <code>/tf_static</code> → 고정된 링크간 변환
구독하는 토픽	없음	<code>/joint_states</code> , <code>/robot_description</code>
Robot Description	<code>robot_description</code> 파라미터(URDF 라인)를 기반으로 Joint 정보 GUI 생성	<code>robot_description</code> 을 기반으로 Link-Joint 구조 파싱, TF 계산 후 <code>/tf</code> 로 Publishing
사용 메시지 타입	<code>sensor_msgs/msg/JointState</code> Header, name, position, velocity, effort	<code>tf2_msgs/msg/TFMessage</code>
세부 내용	- Joint 이름 - 각 조인트 위치, 속도, 힘 - Effort: 조인트에 걸린 힘 - GUI에서 조절 가능	- URDF를 기반으로 링크 간 TF 계산 - Joint 상태를 토대로 3D 위치 관계 계산 - 각 링크에 대응하는 TF 트리 생성

• 실행 흐름도 요약

```

joint_state_publisher → /joint_states → robot_state_publisher
                        ↓
                    TF 계산 (/tf, /tf_static)
                        ↓
                    RViz
  
```

4. Using Xacro

: XML Macro 언어로, URDF 파일을 더 유연하게 작성할 수 있는 도구

- 반복되는 구조, 매개변수 설정 등에 유용
- `.xacro` 확장자 사용
- 매크로 정의 / 변수 사용 / if 조건문 가능
- Xacro → URDF 변환

```
ros2 run xacro xacro ***.xacro > ***.urdf
```

URDF	SRDF	XACRO
Universal Robot Description Format	Semantic Robot Description Format	XML Macro
로봇의 물리적 구조(모델) 정의	로봇의 의미론적(운동학적) 정보 정의	URDF의 확장버전(코드 재사용성 증가)
링크, 조인트, 관성, 충돌 모델 등	운동학 그룹, 충돌 회피 설정, 엔드 이펙터 설정	URDF
Gazebo, Rviz, Moveit!	Moveit!	Gazebo, Rviz, Moveit!
로봇의 구조를 정의하는 기본 파일	URDF를 기반으로 Moveit!을 위한 의미론적 설정을 추가하는 파일	URDF를 생성하는 도구
URDF	URDF를 SRDF로 변환 불가능. 수작업	Xacro파일을 URDF로 변환해야 Gazebo, Rviz, Moveit!등에서 사용가능
URDF 시뮬레이션은 가능하지만 경로계획 수행불가	Moveit!같은 경로 계획 도구 사용하려면 SRDF필요	Xacro는 URDF를 생성하는 도구 실제 사용되는 로봇모델파일은 아님

5. Using URDF with robot_state_publisher

- URDF에서 모델링된 보행 로봇을 시뮬레이션하고 Rviz에서 확인
- 로봇의 링크와 조인트 상태를 기반으로 TF트리를 자동생성
- 구성 흐름

```

URDF (robot_description)
  ↓
joint_state_publisher → /joint_states
  ↓           ↓
robot_state_publisher → /tf + /tf_static
  ↓
RViz (로봇 모델 + 좌표축 시각화)
  
```

• 정리

항목	설명
<code>robot_state_publisher</code>	URDF 구조 → TF 자동 생성
<code>joint_state_publisher</code>	조인트 값 제공 (GUI로 조절 가능)
<code>/tf</code> , <code>/tf_static</code>	RViz, SLAM, Navigation 등에서 필수 사용
<code>robot_description</code>	URDF 텍스트가 저장되는 파라미터 이름

5. 시뮬레이션 개발

5.1 Gazebo

- 로봇 시뮬레이션을 위한 오픈소스 3D 물리 엔진
- ROS2 Humble과 함께 사용하는 Gazebo 버전은 Gazebo-Ignition

```
# Gazebo 설치
sudo apt update
sudo apt install ros-humble-gazebo-ros-pkgs
# Gazebo 실행
gazebo
# ROS2와 Gazebo 연동
gazebo --verbose \
/opt/ros/humble/share/gazebo_plugins/worlds/gazebo_ros_diff_drive_demo.w
orld
```

- Gazebo의 주요 기능
 1. 물리 엔진 제공(ODE, Bullet, DART)
 2. 센서 시뮬레이션
 3. 3D환경
 4. 로봇 모델 시뮬레이션
- **로봇 시뮬레이션** : 컴퓨터 상에서 가상의 환경을 이용하여 로봇의 동작, 센서 데이터, 환경을 개발하고 테스트하는 기술
- Gazebo 시뮬레이션의 구성요소
 - World : 시뮬레이션 환경을 정의하는 파일
 - 로봇 모델 : 로봇 모델을 정의하는 파일
 - 플러그인 : Gazebo에서 제공하는 로봇 제어 API 이용
 - launch.py : 프로젝트 설정하고 실행함

5.2 SLAM

- Simultaneous Localization and Mapping의 약자로 동시적 위치 추적 및 지도 생성
- SLAM 사용 기술
 - Localization : 로봇이 현재 위치를 추정하는 과정

- Mapping : 주변 환경에 대해 지도를 생성하는 과정
- Sensor Fusion : 라이다, 카메라, IMU 센서, GPS, 적외선, 초음파 센서 등 다양한 센서 데이터를 통합하여 정확도를 높이는 기술

5.3 NAV

- Navigation은 SLAM을 활용하여 로봇을 원하는 위치까지 자동으로 도달하게 제어하는 것
- NAV 사용 기술
 - Path Planning : 목표 지점까지 최적의 경로를 찾는 과정
 - Behavior Tree : 로봇의 동작을 트리 형태로 구성하여 유동적으로 관리하고 제어하는 알고리즘
 - Trajectory Tracking : 계획된 경로를 따라 로봇을 제어하는 기술

	SLAM (Simultaneous Localization & Mapping)	Nav2 (Navigation)
역할	맵을 생성하며 로봇의 위치를 추정	이미 존재하는 맵에서 경로를 계획하고 이동
입력 데이터	센서 정보(LiDAR, IMU등)	맵, 로봇 위치, 목표 위치
출력 결과	2D맵, 로봇 위치(/map, /tf)	이동 경로, 속도 명령(/cmd_vel)
실행 시점	맵이 없는 환경에서 최초 탐색에 사용	맵이 준비된 이후 목표지점으로 이동할 때 사용
대표 패키지	slam_toolbox, cartographer, gmapping	nav2_bringup, bt_navigator, planner_server

• SLAM : Visual

- 카메라에서 얻은 2D, 3D 이미지데이터를 사용하여 환경의 고유한 특징점을 추출하는 SLAM
특징점을 다른 프레임(이미지)에서 다시 찾아내고, 이를 비교해 로봇의 상대적 위치를 계산
- 장점: 카메라만으로 저비용 시스템 구현 가능
- 한계: 조명, 기상, 텍스처 없는 평면에서 정확도 감소

• SLAM : LiDAR

- LiDAR에서 거리 데이터를 통해 3D 점 구름(Point Cloud)를 생성하고, SLAM 이전 프레임과 현재 프레임에서 얻은 포인트 클라우드 데이터를 비교하여, 로봇의 상대적 위치와 환경 지도를 동시에 계산
- 장점 : 어두운 환경에서 정확한 거리를 측정

- 한계 : 비용이 높고 외부 노이즈에 민감함

• LiDAR SLAM의 종류

특징	GMapping	Cartographer
호환성	주로 ROS1	ROS1 & ROS2 모두 호환
맵 환경	2D, static한 상황에 유리	2D & 3D
정확도	센서나 Odometry 오류에 민감함	오류에 대해 비교적 강건하고 정확함
로컬라이제이션	대략적인 위치 추정	더 정확한 위치 추정
적용 분야 / 환경	저사양 하드웨어 / 단순한 실내 환경	고정밀 맵이 필요한 복잡한 환경 / 정밀 지도
개발자	 OpenSLAM Give your algorithms to the community	 Google 이 개발한 실시간 SLAM
성능	기본적인 환경에서 충분	복잡하고 넓은 환경에서도 우수

- **SLAM : Sensor Fusion** : 카메라와 라이다 그리고 다른 센서들의 데이터를 융합하여 상호 보완적으로 SLAM을 수행
- **Nav2의 주요 작업을 수행 하는 4가지 서버**
 - Planner Server : Dijkstra나 A*(경로 탐색 알고리즘)등을 사용하여로봇 위치에서 목표 지점까지의최적경로계산
 - Controller Server : DWA 알고리즘을 사용하여 경로를 따라가기 위한 local planning을 구함(cmd_vel생성)
 - Smoother Server : Planner가 생성한 경로를 입력으로 받아, 로봇 주변 환경 정보를 나타내는 costmap을 기반으로 경로를 더 부드럽게 변경
 - Recovery Server : 네비게이션 실패 시 clear costmap이나 회전, 후진 등의 복구 동작을 실행
- **LiDAR 센서의 주요 파라미터**
 - Angle parameters(각도)
 - Time parameters(시간)
 - Range parameters(거리)
- **SLAM : Sensor Fusion**
 - 장점 : 조명 변화, 외부 환경에 더 강건한 시스템
 - 한계 : 데이터 처리량 증가로 실시간 구현 문제

- 대표적인 알고리즘 : Kalman Filter, Particle Filter, Graph-based Optimization, Deep Learning 기반 Fusion

- **Navigation - Nav2의 Behavior Tree 시각화**

```
# 의존성 설치
sudo apt install qtbase5-dev libqt5svg5-dev libzmq3-dev libdw-dev
# Groot 설치
git clone https://github.com/BehaviorTree/Groot.git
cd Groot
git submodule update --init --recursive
mkdir build
cd build
cmake ..
make
```

- **Navigation - Nav2의 Behavior Tree**

- Control node (흐름 제어)
 - Sequence : 하위노드의 동작을 순차적으로 실행, 하나라도 실패하면 **False** 를 반환
 - Fallback : 하위노드의 동작이 실패하더라도 계속 다음 동작을 수행하며 하나라도 **True** 면 **True** 를 반환
- Leaf node (말단 노드 – 실제 동작)
 - Action Node : 동작을 실행 후, 결과 반환 : **True** (동작 완료), **False** (동작 실패), **Running** (동작 수행 중) 등
 - Condition Node : 조건 확인만 수행, 동작 없음, 반환 : true(조건 만족), false(조건 만족하지 않음)

- **Nav2의 주요 작업을 수행 하는 4가지 서버**

- Planner Server : 경로 탐색 알고리즘(Dijkstra, A*)을 이용해 출발점 → 목표 지점 **최적 경로 계산**
- Controller Server : **DWA(Dynamic Window Approach)** 알고리즘으로 로컬 경로 생성 및 **cmd_vel** 출력
- Smoother Server : 경로를 부드럽게 조정, costmap 기반으로 회피 가능한 경로로 보정
- Recovery Server : 경로 실행 실패 시, **회전, 후진, costmap 초기화** 등의 복구 동작 수행

• 3D Camera & SLAM

```
# Gagebo 패키지 설치
sudo apt install ros-humble-gazebo-*
# Turtlebot3 패키지 설치
sudo apt install ros-humble-turtlebot3
sudo apt install ros-humble-turtlebot3-gazebo
# Cartographer 패키지 설치
sudo apt install ros-humble-cartographer
sudo apt install ros-humble-cartographer-ros
# Navigation2 패키지 설치
sudo apt install ros-humble-navigation2
```

• Visual SLAM의 종류

특징	ORB (Oriented FAST and Rotated BRIEF)	RTAB (RealTime Appearance-Based Mapping)
사용 카메라	주로 단안(모노) 카메라 또는 Stereo Camera	RGB-D카메라 혹은 양안(스테레오) 카메라
깊이 감지	깊이 감지 범위가 짧음	깊이 정보를 더 잘 감지함
병렬 처리	작업을 병렬적으로 처리	단계별 수행으로 상대적으로 처리시간이 오래 걸림
맵 생성과 최적화	중요한 위치(키포인트)를 선택, 맵을 만들고 수정	더 밀도 있는 3D 매핑을 수행하여 정확도를 높임
정확도	야외 환경에서 정확도 감소	야외 환경에서도 강건함
기타	정밀한 위치 추정과 맵핑이 필요한 경우	실시간 처리와 대규모 환경에서의 확장성이 중요한 경우
활용사례	연구 개발, 정밀 네비게이션	실시간 로봇 네비게이션, 3D매핑
ROS통합	가능하지만 복잡할 수 있음	매우 용이(ROS2 package제공)하고 주로 많이 사용함

5.4 ROS2 활용 두산 로봇

• 로봇 팔 (Manipulator)

- 인간의 팔 동작과 기능을 모방하도록 설계된 장치
- 관절, 구동 장치 작업 도구로 구성되어 작업을 높은 정밀도와 효율성으로 수행할 수 있음

• 그리퍼

- 로봇팔의 가장 대표적인 작업 도구
- 사람의 손과 유사하며 물체를 쥐고 조작할 수 있음

• 로봇 모션

- **MoveJ** : 각 관절이 **동시에 움직여** 목표 관절 각도로 이동 후 정지
- **MoveL** : 직선 경로를 따라 TCP가 목표 위치까지 이동
- **MovePeriodic** : 일정한 진폭과 주기로 왕복 이동
- **로봇 제어 기초 개념**
 - ACC(가속도) : 로봇 관절의 속도 변화량을 제어하는 파라미터
 - VEL(속도) : 로봇 관절의 속도를 제어하는 파라미터
 - Sync Operation(동기 구동) : 실행 중인 모션 명령어가 완전히 끝난 후 다음 명령어로 이동
 - Async Operation(비동기 구동) : 명령을 실행하자마자 바로 다음 명령 실행 예 , 로봇 arm 이동 중 그리퍼 동작
 - Radius(반경) : 반경(mm)을 설정하여 도착점에 도달하기 전 반경 구간에서는 비동기 구동을 활성화
 - Blending Mode(블렌딩 모드) : 반경(Radius)을 적용했을 때 선행 모션을 유지하여 중첩 실행할 것인지(Duplicate) 선행 모션을 무시하고 덮어씌울 것인지(Override)를 선택하는 옵션
- **Sync vs Async**
 - Sync : 한 번에 하나의 모션 명령어만 수행하는 것으로 기본값으로 설정되어 있음
제어의 예측 가능성이 높음
 - Async : 한 번에 여러 개의 모션 명령어를 수행하는 것으로 모션이 부드럽게 연결됨
동작을 빠르게 수행하여 작업 효율의 증대
But 제어 로직이 복잡해질 수 있음
- **Blending Option : Duplicate vs Override**
 - Duplicate
 - TCP 가 반경에 진입했을 때
 - 선행 모션을 유지하면서 후행 모션을 실행
 - 작업 연속성을 고려한 복잡한 동작 구현에 적합
 - Override
 - TCP 가 반경에 진입했을 때
 - 선행 모션을 덮어쓰는 식으로 후행 모션을 실행
 - 즉각적인 작업전환으로 비상상황 대처에 적합
 - 속도가 중요한 연속작업 (Palletizing, Pick & Place)

- 로봇 외력 (External Force) : 로봇이 환경과 상호작용하거나 외부에서 힘이 가해질 때 로봇에 작용하는 힘이나 토크
 - Compliance Control (순응 제어) : 외력에 순응하며 로봇의 움직임을 조절 용수철 원리
외력이 사라지면 로봇이 원래의 목표 위치로 복귀
 - Force Control (힘 제어) : 로봇이 외력에 대해 평형을 이루도록 힘을 제어
로봇의 자세가 특이점 영역에 들어가면 힘 제어가 불가능할 수 있으므로 자세 변경 필요

6. ROS2와 OpenCV

6.1 ROS2와 OpenCV

- **Opencv** : 컴퓨터 비전과 이미지 처리를 위한 오픈소스 라이브러리
- **Canny Edge**
 - 이미지에서 경계를 찾는 방법으로 , 밝기 변화가 큰 부분을 찾아 edge 로 감지하여 물체의 윤곽을 추출
 - Threshold 는 얼마나 강한 밝기 변화가 있어야 edge 로 인정할지를 결정하는 기준
- **Hough Line Transform**
 - 직선의 방정식을 만들어서 그 직선을 지나는 점의 개수가 Threshold 임계값 을 넘기면 직선으로 판단함
 - 수학적 형태를 이루는 점들의 집합을 찾는 방법
 - 점이 흩어져 있는 복잡한 이미지에서 찾고 싶은 패턴 직선 , 원 , 타원 을 수학적으로 그 패턴을 만족하는 점들을 모아서 찾는 방법
- **Hough Circle Transform**
 - 이미지에서 edge 를 검출하고 각 edge 에 속한 점들에 대해 가능한 모든 원의 중심과 반지름을 계산
 - 원의 중심으로 검출된 수가 threshold 를 넘기면 그 좌표에서의 값이 원의 중심과 반지름이 됨

```
# ros2_ws 디렉토리로 이동
cd ros2_ws
# opencv.zip파일의 압축을 해제 후 /src와 /img파일을 ros2_ws로 옮기기
unzip opencv.zip
colcon build
```

```
source install/setup.bash
```

```
# 터미널1
```

```
ros2 run hough_transform image_publisher --ros-args -p image_path:=img/sudoku.png
```

```
# 터미널2
```

```
ros2 run hough_transform hough_transform --ros-args -p method:=line
```

```
# 터미널3
```

```
rqt
```

```
# circle 감지
```

```
# 터미널1
```

```
ros2 run hough_transform image_publisher --ros-args -p image_path:=img/coin.png
```

```
# 터미널2
```

```
ros2 run hough_transform hough_transform --ros-args -p method:=circle
```

```
# 터미널3
```

```
rqt
```

6.2 ROS2와 차선인식

- **차선인식** : 차선 인식은 도로상의 차선을 감지하고 추적
- **사용 기술**
 - 이미지 전처리 : 이미지를 차선 감지에 적합하게 보정 노이즈 제거 , 색상 변환 , 관심 영역 설정
 - 슬라이딩 윈도우 기법 : 이미지 상에서 연속된 영역을 이동하며 차선을 찾고 , 연결하여 차선을 추정

```
# ros2_lane_ws.zip파일 압축 해제
```

```
unzip ros2_lane_ws.zip
```

```
# 패키지 빌드
```

```
colcon build
```

```
source install/setup.bash
```

```
# 터미널1
```

```
ros2 run lane_detect publisher_node --ros-args -p video_path:=/video/track_video_1.mp4
```

```
# 터미널2
```

```
ros2 run lane_detect subscriber_node
```

```
# 터미널3  
rviz2
```

```
# camera_processing.py  
def process_image(self, img): # 차선 검출 전처리  
    if img is None:  
        return None # 입력 이미지가 None이면 처리하지 않고 반환  
    img = self.remove_lighting(img) # 흑백 전환, 조명 제거  
    # 이진 변환(배경과 흰색 차선 분리)  
    _, img = cv2.threshold(img, self.bin_threshold, 255, cv2.THRESH_BINARY)  
    img = cv2.medianBlur(img, self.MedianBlur) # 중간값 블러로 노이즈 제거  
    img = self.warp(img) # Warp 변환으로 차선을 평행하게 만들기  
    filtered, img = self.choose_filtered_img(img) # 차선 곡률 파악  
    return img, filtered # 최종 이미지와 필터 결과 반환
```

```
# slide_window.py  
'''  
1. 이미지 ROI 설정  
2. 슬라이딩 윈도우 파라미터 설정  
3. 슬라이딩 윈도우 순회 탐색  
4. 슬라이딩 윈도우에서 차선을 찾은 경우  
5. 슬라이딩 윈도우에서 차선을 못 찾은 경우  
6. 수직 방향으로 차선 감지 및 시각화  
'''
```

7. ROS2 시각화

- **Open3D** : 컴퓨터 그래픽과 3D 데이터 처리 작업을 위한 Python 라이브러리
- **3D 데이터 처리**
 - 인간이 물체를 보는 ' 방식과 이를 이해하고 분석하는 '과정을 모방함
 - 센서나 스캐너를 통해 3D 포인트 클라우드 데이터를 받아서 전처리 분석 시각화를 수행함
 - 자율 주행 , 로봇 공학 , AR/VR 등 다양한 분야에서 활용됨
- **PointCloud**
 - LiDAR 센서 , RGB D 센서 등으로 부터 수집되는 데이터

- 물체에 빛 신호를 보내서 돌아오는 시간을 기록하여 각 빛 신호당 거리 정보 계산하고 하나의 점 (를 생성
- 3차원 공간상에 퍼져 있는 여러 포인트 (의 집합 (을 의미 . (x, y, 의 3 차원 정보
- 2D 이미지와는 다르게 깊이 (Z 축 정보를 가지고 있으며 Nx3 의 numpy 배열로 표현
각 n 줄은 하나의 점과 Mapping

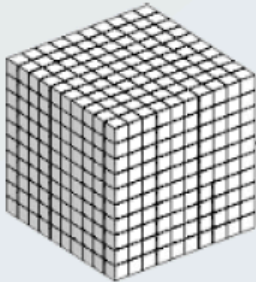
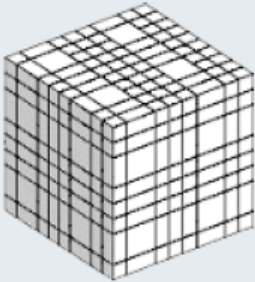
```
# Terminal 1
colcon build
source install/setup.bash
pip install open3d
# fragment.ply 파일의 경로와 voxel_size를 옵션으로 전달
ros2 run point_cloud pcd_publisher_node ~/ros2_ws/src/point_cloud/resource/fragment.ply 0

# Terminal2
ros2 run point_cloud pcd_subscriber_node
```

• Rviz2로 시각화하기

```
# voxel_size 를 0.05 로 설정하면 듬성듬성 랜더링되는 3D 모델을 볼 수 있다
ros2 run point_cloud pcd_publisher_node ~/ros2_ws/src/point_cloud/resource/fragment.ply 0
rviz2
```

- **.ply 파일** : 3D 객체의 모양을 저장하는 파일 형식, 점, 면 등의 정보를 포함해 3D 데이터를 저장
- **Voxel** : 'Volume'과 ' ' 의 합성어로 , 3 차원 공간에서의 픽셀을 의미

Voxel의 크기가 작을 때	Voxel의 크기가 클 때
	
해상도 높음	해상도 낮음
처리 속도 느림	처리 속도 빠름

- **행렬의 회전** : 3차원 모델을 회전시킬 때 행렬의 곱셈을 통해 구현

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \times \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- 90° 만큼 회전시킬 때

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} \times \begin{bmatrix} \cos 90 & -\sin 90 \\ \sin 90 & \cos 90 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

- 3차원에서 회전 행렬

$$\begin{array}{ccc}
 \text{x축이 고정된 회전} & \text{y축이 고정된 회전} & \text{z축이 고정된 회전} \\
 R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix} & R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix} & R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}
 \end{array}$$

• PCD 코드 설명

pcd_publisher_node.py	pcd_subscriber_node.py
pcd 파일을 읽어옴	pcd Topic 을 구독
Numpy로 변환 , 가공 회전 , Z 축 이동	PointCloud2메시지를 parsing 해서 numpy 로 복원
PointCloud2 메시지 생성 및 Publishing	복원된 numpy array 를 Open3D 로 시각화
30Hz 주기로 Publishing	새로운 프레임이 들어오면 렌더링 업데이트